Introduction to Embedded Systems

Course Code: 244066

Lab 5

STM32 Timer Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Instructions

- Complete every task in the lab practice session.
- For each task, include a 2–5 line description along with a screenshot of both your code and its output.
- Submit your lab report by email to Imran.Nadeem@outlook.com, using the subject line format as:
- **ES24-[lab number]-[registration number]**.
- Lab report need to be submitted before **11:59 pm**.
- Submissions received after the deadline will not be accepted.

Overview

Overview of Learning Outcomes

By completing these simulated labs, you will:

- Correct configuration of timers and proper handling of interrupts.
- Well-commented code and clear explanations of how the timers are configured.
- Correct simulation output (LED blink, PWM, etc.) in Keil MDK.

Setup Notes

- Use the same project template as the earlier labs: device **STM32F103C4**, **CMSIS Core** + **Device Startup**, #include "**stm32f10x.h**" (plus <**stdint.h**>), Debug tab set to **Use Simulator**, and the same **STMaccess.ini** (MAP 0x40000000,0x400FFFFF READ WRITE).
- Set **Xtal (MHz)** to **72** in Options for Target → Target, so the simulated time matches the **chapter's** 72 MHz calculations. Set the optimization level to **-O0**.
- The **STM32F103C4** is a low-density device with TIM1, TIM2 and TIM3 only (no TIM4). These labs use TIM2 and TIM3 instead of the chapter's TIM4 example.
- **Verify** results in the Watch window (**Hex format**). Measure elapsed simulated time with Registers window → Internal → Sec (and States), as in Chapter 3.
- Reuse your **PC13 LED** helper from **Chapter 3** (clock enable, push-pull output, toggle via BSRR).
- The **simulator** may not model every timer or **watchdog**. If a register does not change by itself, force the status bit in the Watch window (for example TIM2->SR = 1). Pend interrupts manually with NVIC->ISPR[0] (TIM2 is IRQ 28 → 0x10000000, TIM3 is IRQ 29 → 0x20000000).

Task 1: SysTick Polling Delay and 500 ms LED Blink

Steps

1. Calculate the LOAD value for a 1 ms tick at 72 MHz (72000000 / 1000 - 1).
2. Configure SysTick: SysTick->LOAD = 71999;, SysTick->VAL = 0;, SysTick->CTRL = 5; (enable + CPU clock, no interrupt).
3. Note that the chapter example uses CTRL = 7, which also enables the interrupt. If you use 7, you must define SysTick_Handler. Without it, the default handler in the startup file loops forever. For polling, use 5.
4. Write delay_ms(uint32_t ms) that loops ms times, each time waiting for COUNTFLAG (bit 16 of SysTick->CTRL).
5. In main(), toggle the PC13 LED and call delay_ms(500) in an infinite loop.
6. Add SysTick->LOAD, SysTick->VAL, SysTick->CTRL and GPIOC->ODR to the Watch window. Verify that LOAD reads 0x0001193F.
7. Set a breakpoint after each delay_ms(500) call. Check that bit 13 of GPIOC->ODR toggles and record Sec at each stop. The difference should be about 0.5 s.
8. SysTick is a 24-bit counter (maximum 0xFFFFFF). Calculate the LOAD that a single 500 ms delay would need and the longest single period possible at 72 MHz (about 233 ms). Explain in 2 sentences why delay_ms counts 1 ms ticks in a loop.
9. If the program hangs waiting for COUNTFLAG, set SysTick->CTRL to 0x00010005 in the Watch window to simulate the flag.

Task 2: Timer Delays with TIM2: PSC and ARR Calculations

Steps

1. Using f = f_clock / ((PSC + 1) × (ARR + 1)), fill in a table at 72 MHz for these cases: (a) PSC = 71, ARR = 999; (b) PSC = 7199, ARR = 9999; (c) PSC = 7199, ARR = 4999. For each, give the update frequency and period.
2. Find values for a 2-second delay. Check that both PSC and ARR fit in 16 bits (maximum 65535).
3. Enable the TIM2 clock: RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;.
4. Configure case (b): TIM2->PSC = 7199;, TIM2->ARR = 9999;.
5. Load the prescaler by generating an update event: TIM2->EGR = 1;. Then clear the flag with TIM2->SR &= ~1;. Without this, the first period would use the old prescaler value.
6. Start the timer with TIM2->CR1 |= 1; (CEN). In the main loop, wait for UIF (while(!(TIM2->SR & 1));), clear it, and toggle the LED.
7. Add TIM2->PSC, TIM2->ARR, TIM2->CNT, TIM2->SR and TIM2->CR1 to the Watch window. Verify that CNT counts up, resets at overflow, and that Sec between toggles is about 1 s.
8. Change to case (c) and confirm the 0.5 s period.
9. Extension: set the DIR bit (bit 4 of CR1) and record how CNT behaves in down-count mode compared with up-count mode.

Task 3: PWM LED Brightness Fade with TIM3

Steps

1. Enable the GPIOA and TIM3 clocks. Configure PA6 (TIM3 CH1) as alternate-function push-pull, 50 MHz: GPIOA->CRL &= ~(0xF << 24); GPIOA->CRL |= (0xB << 24);.
2. Set a 1 kHz PWM period: PSC = 71, ARR = 999.
3. Configure channel 1 for PWM mode 1: TIM3->CCMR1 = 0x68; (OC1M = 110, OC1PE = 1). Set CCR1 = 250. Enable the output with TIM3->CCER |= 1;.
4. Generate an update event (EGR = 1) and start the timer (CR1 |= 1). Calculate the duty cycle (CCR1 / (ARR + 1), which should be 25%).
5. Add TIM3->CNT, TIM3->CCR1, TIM3->ARR and TIM3->CCER to the Watch window. Halt at three different breakpoints. At each one, use the chapter rule "output HIGH while CNT < CCR1" to predict whether PA6 is HIGH or LOW, and record CNT and CCR1.
6. Reuse the 1 ms SysTick from Task 2. Every 10 ms, increase CCR1 by 10 up to 999 and then decrease it back to 0, to create a fade effect. Record CCR1 at 10 evenly spaced stops.
7. Calculate the duty cycle for CCR1 = 100, 500 and 900, and note how perceived brightness changes.
8. If the Logic Analyzer window is available (View → Analysis Windows), add PA6 and compare the pulse width at two different CCR1 values.
9. Extension: set CMS = 01 (bit 5 of CR1) for center-aligned mode. Predict how the PWM frequency changes for the same ARR, and check by measuring CNT behavior.

Task 4: Independent Watchdog (IWDG): Feeding and Timeout

Steps

1. Reuse your LED blink from Task 1 (delay_ms(500) and a toggle).
2. Enable the LSI clock: RCC->CSR |= RCC_CSR_LSION; and wait for LSIRDY (bit 1). If the program hangs, set bit 1 of RCC->CSR in the Watch window.
3. Unlock and configure: IWDG->KR = 0x5555;, IWDG->PR = 4; (divide by 64), IWDG->RLR = 1000;. Wait until IWDG->SR is 0.
4. Start the watchdog with IWDG->KR = 0xCCCC;.
5. Calculate the timeout RLR × 64 / f_LSI for LSI = 30, 40 and 60 kHz. Compare with the chapter's "~1 s" claim and explain how the LSI tolerance affects the feeding interval you should choose.
6. Run A (with feed): in the main loop, toggle the LED, call delay_ms(500), then write IWDG->KR = 0xAAAA;. Set a breakpoint on the first line of main() and confirm it is hit only once.
7. Run B (no feed): comment out the 0xAAAA line and rebuild. On hardware the MCU would reset after the timeout and the LED pattern would restart. Check whether the breakpoint at the start of main() is hit again.
8. Fallback if the simulator does not model IWDG: build a software model. Use a volatile uint32_t wd_counter = 1600; decremented in your SysTick ISR every millisecond. Reset it to 1600 in a feed() function, and call NVIC_SystemReset() when it reaches 0. Run with and without feed(), and note the behaviour.
9. In 2–3 sentences, explain why the IWDG runs on the LSI clock and cannot be stopped once started.

Task 5: Window Watchdog (WWDG): Refreshing Inside the Window

Steps

1. Enable the WWDG clock: RCC->APB1ENR |= (1 << 11);.
2. With PCLK1 = 36 MHz and WDGTB = 3, calculate the time per counter step (4096 × 8 / 36 MHz, about 0.91 ms). Using a start value of 0x7F, calculate the maximum timeout (64 steps, about 58.2 ms).
3. Choose the window value W = 0x50. A refresh is only allowed when the counter is at or below W. Calculate the earliest allowed refresh time (47 steps, about 42.8 ms). Write both times in a small timeline table.
4. Configure the window: WWDG->CFR = (3 << 7) | 0x50;. Verify that it reads 0x000001D0.
5. Start the watchdog: WWDG->CR = 0x80 | 0x7F;. Verify that it reads 0x000000FF.
6. Write uint8_t wwdg_refresh(void). If (WWDG->CR & 0x7F) <= 0x50, write WWDG->CR = 0xFF;, increment refresh_ok and return 1. Otherwise, increment too_early and return 0 without refreshing.
7. Test three cases by editing the counter in the Watch window (if the simulator does not decrement it by itself): WWDG->CR = 0xFF (T = 0x7F, too early), WWDG->CR = 0xCA (T = 0x4A, inside the window), and WWDG->CR = 0xBF (T = 0x3F, hardware would reset at this point).
8. For each case, record the counter, the result of wwdg_refresh(), and the expected hardware behaviour in a table.
9. In 3 sentences, compare WWDG with IWDG: clock source (APB1 vs LSI), the “too early” and “too late” rule, and when each is the better choice.